Scientific Computing: Exercises

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB

Chair of *Software and Algorithms for Discrete Optimization* at TU Berlin

L03

Working with data

1. The following binary numbers presents positive integers (i64).
   - 1011001_2
   - 0110011_2
   - 1010010000010001_2
   - $1101111001100 11_2$

   a) Convert the binary numbers to their decimal values (show your work)
   b) Find the two's complement representation of those binary numbers

2. Patterns
   a) How many different bit patterns can be represented using 63 bits?
   b) If we want to store any integer x where $0 \leq x \leq 25$.
      What is the smallest number of bits we can use?
   c) If we represent colours as red-green-blue (RGB) triples, where we use 4 bits for red, 4 bits for green, and 4 bits for blue. How many different colours can we represent?
   d) If we represent a decimal number (like 13579) by concatenating the binary representations of each decimal digit. How many bits would we need for a number which contains n digits?

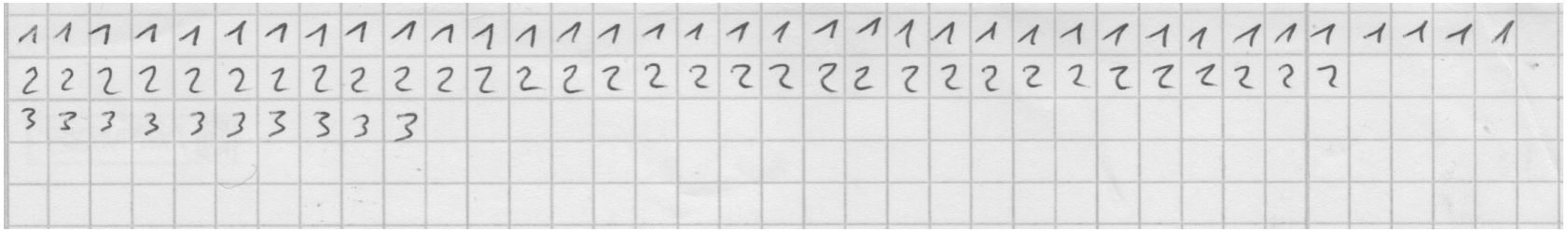
3. Find out which rounding mode Rust is using for floating point computations.

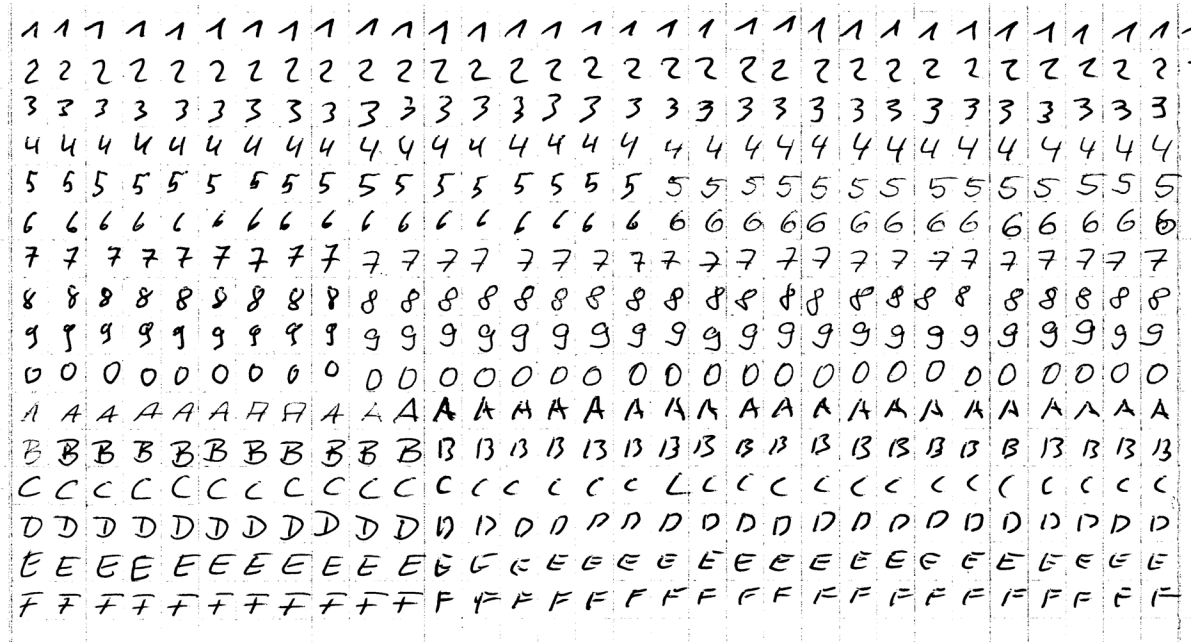
4. If we have a 8-bit IEEE754 type floating point number (E4M3: sign, 4-bit exponent, 3-bit mantissa) which values can we represent?

Exercise 03 – Programming

1. Take a sheet of graph paper (Karopapier) with at least 30 squares per row.
2. Then write for each of the characters 1,2,3,4,5,6,7,8,9,0,A,B,C,D,E,F (in this order) one row with 30 times that character.
   You should then have n = 16 × 30 = 480 characters.
3. Scan the paper. Recommended is black and white using 600 dpi resolution.
   If you don't have a scanner, use the best phone you have or the best camera.
   If possible, use PNG or TIFF file format to save.
   If you can only use JPEG, try the lowest compression = best quality. Find out why JPEG is not recommended here.
4. Convert the image you have to the PBM (portable bitmap https://netpbm.sourceforge.net/doc/pbm.html, or https://en.wikipedia.org/wiki/Netpbm#File_formats) file format. This can be done, e.g., with utilities from https://netpbm.sourceforge.net
5. Write a Rust program that reads the PBM file of the image of the page and writes out a unique PBM file for each character on the sheet, i.e., write n files with one character each. Don't use any extra crates/libraries outside std:: .
   Note: You can read in either P1 (text) or P4 (binary) format.
   However, if you read P1, make sure you cannot only read files you produce but all valid P1 format image files.
6. Each file should be in PBM format with a size of 128 x 128 and only contain one character.
7. The file should be named: `CXX.pbm`, where C is the character, and X, is 01..30 the repetition of the character, e.g, `517.pbm`, saying it is the 17th five on the paper. All the files should be put into a folder named `output/` .
8. For each character compute the geometric mean of the number of black pixels over all copies of that character and print them one per line as `C: GM` with C being the character and GM the geometric mean.
9. **Commit all the relevant files including the scan and the resulting images.**
   Submit also the physical written page during the lecture or exercise.

Example





600 dpi black and white

PBM Portable Bitmap Format

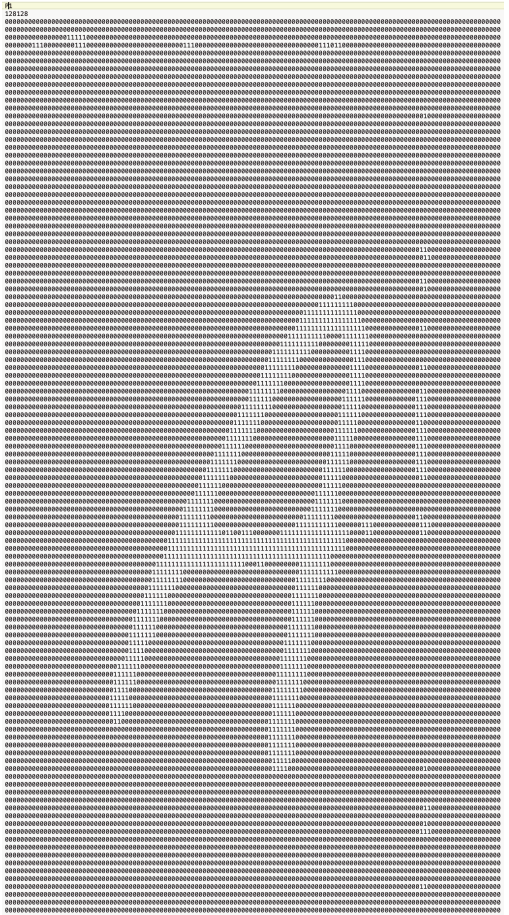
There is the binary version “P4”, and the text version “P1”.

The binary version is certainly more clearly defined.

In the pic on the right, there where spaces between the digits.

```
P1
# This is an example bitmap of the letter "J"
6 10
0 0 0 0 1 0
0 0 0 0 1 0
0 0 0 0 1 0
0 0 0 0 1 0
0 0 0 0 1 0
0 0 0 0 1 0
1 0 0 0 1 0
0 1 1 1 0 0
0 0 0 0 0 0
0 0 0 0 0 0
```

```
P1
# This is an example bitmap of the letter "J"
6 10
000010000010000010000010000010000010100010011100000000000000
```



A collection of lints to catch common mistakes and improve your Rust code.

https://doc.rust-lang.org/nightly/clippy/index.html

USE IT!

```
$ cargo clippy
```